Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

August 14, 2026

Contents

Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the fundamental problem is regular expression matching: given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression `(ab|a)*` matches the input string `"aba"`, as it can be decomposed into `"ab"` followed by `"a"`, but does not match `"abb"`, since no sequence of subexpressions can produce that input.

For many applications, a Boolean answer is sufficient. An input validator, for instance, may only need to accept or reject a password, URL, or configuration value. Other applications require more: a lexer must determine how an input decomposes into tokens, while capturing groups, schema validation, and information extraction require identifying which parts of the input correspond to different parts of the expression. In these settings, a matcher must recover the *structure* of a match, and the problem shifts from recognition to parsing.

Efficiently solving these problems remains an active area of research: the choice of algorithm directly influences performance, memory consumption, and security, and, once matching becomes parsing, the choice of how a match's structure is recovered directly influences correctness as well.

Existing approaches include backtracking, finite-automata-based techniques such as nondeterministic (NFA) and deterministic (DFA) automata, and derivative-based algorithms. Each approach provides different trade-offs between implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely adopted, but may exhibit exponential running time on carefully constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require costly preprocessing and can suffer from state-space explosion.

Derivative-based algorithms provide an alternative by transforming the regular expression itself as input symbols are consumed. By maintaining a direct correspondence between an expression and its successive residuals, they offer an elegant foundation for structured regular expression matching.

However, derivative-based techniques are less commonly used in practical regular expression tools than other matching approaches. Bridging this gap between theory and practice motivates the work presented in this thesis.

1.2 Problem Statement

Brzozowski introduced regular expression derivatives as an algebraic characterization of regular expression matching [Brzozowski, 1964]. Antimirov later introduced partial derivatives, which represent sets of residual expressions and provide a construction closely related to nondeterministic automata while reducing redundancy compared with ordinary derivatives [Antimirov, 1996]. Building on these ideas, Sulzmann and Lu presented a derivative-based algorithm for POSIX-correct regular expression parsing [Sulzmann and Lu, 2014].

Although the algorithm is formally developed and accompanied by a Haskell reference implementation, its practical behaviour can be further studied through an implementation in a systems programming language. Rust provides such an environment through its compile-time ownership model, allowing memory allocation, stack usage, and execution time to be studied as direct properties of the implementation.

This thesis investigates the practical implementation of Sulzmann and Lu's POSIX-correct derivative-based parsing algorithm in Rust and explores implementations based on Antimirov's partial derivatives and the bit-coded optimization proposed by the authors. The resulting implementations are then evaluated in terms of performance and resource usage and compared against established regular expression implementations, including Rust's `regex` crate and Google's RE2.

Furthermore, the derivative-based representation provides an opportunity to show the reasoning of the algorithm. This thesis therefore also shows how derivative computations can be used as a basis for diagnostic tooling, allowing matches and mismatches to be explained through the transformations performed during the matching process.

1.3 Contributions

This thesis makes the following contributions:

- **A Rust implementation of Sulzmann and Lu's derivative-based POSIX parser**, including a direct two-pass construction following the original algorithm and a bit-coded single-pass optimization. The implementation additionally explores partial derivatives as an alternative representation.
- **An empirical evaluation** comparing derivative-based and partial-derivative-based implementations with each other and with established regular expression engines, including Rust's `regex` crate and Google's RE2.
- **A lightweight derivative-tracing mismatch diagnostics system**, exposing the parser's intermediate derivative computations and using them to explain successful matches and failures.

1.4 Thesis Outline

The remainder of this thesis is structured as follows. Section 2 introduces the syntax and semantics of regular expressions, together with the Rust representation of their abstract syntax used throughout the rest of this work. Section 3 builds on this representation to introduce Brzozowski's derivatives and Antimirov's partial derivatives as algorithms for deciding language membership, along with their Rust implementation – the foundation on which the parsers presented later are built.

Before extending these algorithms to parsing, Section 4 distinguishes matching from parsing and introduces the disambiguation problem that arises once a parse tree, not just a boolean, is required. Section 5 then presents the derivative-based POSIX parser adopted as this thesis's algorithmic basis, Sulzmann and Lu's construction [Sulzmann and Lu, 2014], together with its Rust implementation in both a direct ("inj") and a bit-coded form. Section 6 extends this construction from Brzozowski derivatives to Antimirov's partial derivatives, again in both its direct and bit-coded variants.

With all four parser variants in place, Section 7 evaluates their correctness and performance against each other and against established regular expression engines. Section 8 situates this work relative to related approaches, reflects on translating the underlying algorithm from Haskell to Rust, and discusses the derivative-tracing facility built alongside the parser, including its levels of diagnostic detail and their usefulness. Section 9 closes by summarizing what these four implementations and their evaluation establish, and outlines the extensions – including a fuller mismatch diagnostics system built on this tracing facility, and the algorithmic extensions suggested but not implemented by Sulzmann and Lu left as future work.

Regular Expressions

2.1 Syntax

A regular expression is built from a small set of primitive constructs, combined by three operators. Over an alphabet Σ of symbols, the syntax is given by the following grammar:

$$r, s \;::=\; x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \qquad (x \in \Sigma)$$

Here x denotes a literal symbol, ε the expression matching only the empty word, and $\emptyset$ the expression matching no word at all. The operators $+$, $\cdot$, and * denote alternation, concatenation, and repetition (Kleene star) respectively; concatenation is frequently left implicit, writing rs for $r \cdot s$.

As a running example, consider the problem of recognizing binary strings that end in `"01"` – a small but non-trivial pattern that touches every construct in the grammar above. Over $\Sigma = \{0, 1\}$, this pattern is expressed as

$$r_{01} \;=\; (0 + 1)^* \cdot 0 \cdot 1.$$

Reading this expression against the grammar: $(0 + 1)$ is an alternation of two literals, $(0 + 1)^*$ applies Kleene star to that alternation, and the result is concatenated with the literals 0 and 1 in sequence. This example is used throughout the remainder of this section.

2.2 Semantics

The syntax above describes the *shape* of an expression; the semantics assigns each expression the *language* it denotes. For a regular expression r, write $L(r)$ for the set of words it matches. The semantics is defined by structural recursion on r:

$$\begin{aligned}
L(x) &= \{x\} \\
L(\varepsilon) &= \{\varepsilon\} \\
L(\emptyset) &= \{\} \\
L(r + s) &= L(r) \cup L(s) \\
L(r \cdot s) &= \{vw \mid v \in L(r),\ w \in L(s)\} \\
L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid w_1, \ldots, w_n \in L(r),\ n \geq 1\}
\end{aligned}$$

A word w *matches* r exactly when $w \in L(r)$ – this is the membership problem introduced in Section 1.1, now stated precisely.

Unwinding the semantics for the running example, $L(0+1) = \{0, 1\}$, so $L((0+1)^*)$ is the set of all finite binary strings, including the empty string. Concatenating with $L(0) = \{0\}$ and then $L(1) = \{1\}$ restricts this to

$$L(r_{01}) = \{\, w \cdot 0 \cdot 1 \mid w \in \{0,1\}^* \,\},$$

i.e. exactly the binary strings ending in `"01"` – `"01"`, `"001"`, `"1101"`, and so on, but not `"010"` or the empty string. This last point is worth making explicit: r_{01} is *not* nullable – $\varepsilon \notin L(r_{01})$, since every word in $L(r_{01})$ must end in the literal 1. Nullability, i.e. whether $\varepsilon \in L(r)$, is a property later chapters compute directly on the syntax of r rather than by reasoning about $L(r)$; the two are guaranteed to agree by the semantics defined here.

2.3 Rust Implementation

The grammar above is represented directly as a Rust enum, mirroring each syntactic construct one to one:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi,
    Eps,
    Lit(char),
    Seq(Box<Regex>, Box<Regex>),
    Alt(Box<Regex>, Box<Regex>),
    Star(Box<Regex>),
}
```

This one-to-one correspondence is not incidental. Rust's enums are algebraic data types: each variant corresponds to exactly one production of the grammar, and Rust's `match` requires every variant to be handled, rejecting the program at compile time otherwise. This gives the same guarantee an inductive definition of r in Section 2.1 carries on paper – that a function defined by cases on r is defined for every possible r – as a property the compiler checks, rather than one that must be checked by inspection. This is a structural fit shared with other languages that support algebraic data types, including Haskell itself; Rust's specific contribution is not this correspondence but what it costs to maintain, which is not always nothing. `Phi` and `Eps` carry no data, matching $L(\emptyset) = \{\}$ and $L(\varepsilon) = \{\varepsilon\}$ directly, and `Lit` carries the literal symbol x – these three variants transcribe directly with no adjustment. The recursive constructs – `Seq`, `Alt`, `Star` – do not: unlike Haskell's `data R = ... | Seq R R | ...`, whose recursive constructors are represented indirectly by the runtime with no annotation required, Rust must know the exact size of a type at compile time, and a `Regex` containing a `Regex` directly would have unbounded size. `Box<Regex>` breaks the recursion by storing the subexpression on the heap and holding only a pointer inline – the first of several places in this thesis where a construct free in the Haskell reference requires an explicit, visible workaround in Rust, revisited more generally in Section 8.2.

Derivatives and Partial Derivatives

Chapter 2 fixed the syntax and semantics of regular expressions: $L(r)$ tells us, declaratively, which words r matches. It says nothing about how to compute membership. This chapter introduces three ways of doing so – naive recursive backtracking, and two algebraic reformulations due to Brzozowski and Antimirov – each realized as a Rust matcher with the common signature `fn(&str, &Regex) -> bool`. All three answer exactly the membership question of Section 2.2; they differ in how directly that question is asked of the input, and in what it costs to ask it repeatedly. The derivative operation introduced here is also the foundation the POSIX parsers of Chapters 5 and 6 are built on – membership is a special case of parsing that discards the parse tree, and it is the simplest setting in which to introduce the operation itself.

3.1 Deciding Membership: An Overview

The most direct way to decide $w \in L(r)$ is to read $L(r)$'s definition as an algorithm: try every way w could have been produced by r's structure, and succeed if any of them works. Writing w as an indexable sequence of characters and (i, j) for the substring between positions i and j, membership of the whole word corresponds to $(0, |w|)$, and each case of $L(r)$ becomes a recursive case of a function `matchR` deciding whether r matches the substring (i, j):

- ε matches (i, j) iff $i = j$.
- A literal x matches (i, j) iff $j = i + 1$ and the character at position i is x.
- $r + s$ matches (i, j) iff r matches (i, j) or s does.
- $r \cdot s$ matches (i, j) iff there is some split point $k \in [i, j]$ such that r matches (i, k) and s matches (k, j).
- r^* matches (i, j) iff $i = j$, or there is some $k \in (i, j]$ such that r matches (i, k) and r^* matches (k, j).

This is a correct matcher by construction: every case reproduces the corresponding case of $L(r)$ verbatim, so there is nothing to prove beyond the definition itself. It is also, in general, an expensive one, because concatenation and star both branch over *every* split point in the current range rather than committing to one, and nothing remembers a sub-range once it has been explored from a different call.

To see the blowup concretely, take $r = a \cdot a \cdot a$ (three literals concatenated, right-associated as $a \cdot (a \cdot a)$) against $w = \texttt{aaa}$. Matching $(0, 3)$ against r tries every split point $k \in \{0, 1, 2, 3\}$

for the outer concatenation, but only $k = 1$ can possibly succeed, since the left operand is a single literal and can only match a range of length exactly 1; the other three attempts are wasted work discovered only after descending into them. The right operand at that point, $(1, 3)$ against $a \cdot a$, repeats the same pattern one level down, again trying every split point in $\{1, 2, 3\}$ before finding $k = 2$. Each concatenation node re-explores every split point of the range handed to it with no memory of the fact that the same range, or a sub-range of it, may already have been tried while resolving a sibling call – for a chain of n literals against an input of length n, the number of (i, j) pairs examined this way grows exponentially in n rather than linearly. `src/matchers/match_naive.rs` transcribes this scheme directly, operating on a `Vec<char>` and a pair of `usize` bounds in place of the Haskell reference's `matchR w (i,j) r`; the benchmarks in Chapter 7 restrict this matcher to inputs of length 15 or less precisely because of this blowup.

`match_naive` is nonetheless useful, and stays in the codebase for that reason: because it is a direct transcription of $L(r)$'s definition, it serves as an independent oracle that the more efficient matchers introduced below can be checked against – any disagreement is a bug in one of the newer matchers, not a question of what the "correct" answer is supposed to be. The remainder of this chapter develops two such matchers, both based on a different idea entirely – instead of searching for a decomposition of w against a fixed r, *transform* r itself as each character of w is consumed, so that membership of the remaining suffix is asked against a smaller, adjusted expression rather than re-asked against the same r every time.

3.2 Brzozowski Derivatives

Brzozowski's derivative of r with respect to a symbol x, written $r\backslash x$, is the expression describing exactly what remains of r's language after the leading symbol x has been removed:

$$L(r\backslash x) \;=\; x\backslash L(r) \;=\; \{\, w \mid xw \in L(r) \,\}[\textit{Brzozowski}, 1964].$$

Deciding whether a word $x_1 x_2 \cdots x_n$ is in $L(r)$ then reduces to computing the chain $r\backslash x_1\backslash x_2\backslash \cdots \backslash x_n$ and checking whether the final expression accepts the empty word – i.e. whether it is *nullable*. Both nullability and the derivative itself are defined by structural recursion directly on the syntax of r, with no reference to $L(r)$ needed once the base cases are fixed:

$$\begin{aligned}
n(\emptyset) &= \text{false} & n(\varepsilon) &= \text{true} & n(x) &= \text{false} \\
n(r + s) &= n(r) \vee n(s) & n(r \cdot s) &= n(r) \wedge n(s) & n(r^*) &= \text{true}
\end{aligned}$$

$$\emptyset\backslash x = \emptyset \qquad \varepsilon\backslash x = \emptyset \qquad y\backslash x = \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases}$$

$$(r + s)\backslash x = r\backslash x + s\backslash x \qquad r^*\backslash x = (r\backslash x) \cdot r^*$$

$$(r \cdot s)\backslash x = \begin{cases} (r\backslash x) \cdot s \;+\; s\backslash x & n(r) \\ (r\backslash x) \cdot s & \text{otherwise} \end{cases}$$

The only case that needs justification beyond direct case analysis on $L(r \cdot s)$ is concatenation: if r is nullable, a word starting with x can either continue matching r (so the

derivative goes into r while s is left untouched) or arise from r already being satisfied by ε and x belonging to s instead – hence the extra $s\backslash x$ summand exactly when $n(r)$ holds. If r is not nullable, the second possibility cannot arise, since $\varepsilon \notin L(r)$ rules out r having already been satisfied before any input was consumed.

Worked example: single-symbol matching. Consider x^* against the word `xx`. Writing $r \xrightarrow{x} r\backslash x$ for one derivative step,

$$x^* \xrightarrow{x} (x\backslash x) \cdot x^* = \varepsilon \cdot x^* \xrightarrow{x} (\varepsilon\backslash x) \cdot x^* + x^*\backslash x = \emptyset \cdot x^* + (x\backslash x) \cdot x^* = \emptyset \cdot x^* + \varepsilon \cdot x^*.$$

Checking nullability of the final expression by structural recursion,

$$\begin{aligned} n(\emptyset \cdot x^* + \varepsilon \cdot x^*) &= n(\emptyset \cdot x^*) \vee n(\varepsilon \cdot x^*) \\ &= (n(\emptyset) \wedge n(x^*)) \vee (n(\varepsilon) \wedge n(x^*)) \\ &= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) \;=\; \text{true}, \end{aligned}$$

so x^* matches `xx`. Two things are worth noting in this short trace. First, the expression has grown from one node to five across two steps, purely from the $+$ introduced at each nullable-prefix step of the concatenation rule; nothing above bounds this growth, and a third derivative step would nest a further $+$ inside each existing summand. Second, applying the algebraic identities $\emptyset \cdot r = \emptyset$ and $\emptyset + r = r$ collapses the final expression back down to $\varepsilon \cdot x^*$ – exactly the expression reached after only *one* derivative step. This is not a coincidence particular to this example: it is the general phenomenon Section 5.4 returns to for the bit-coded construction, where the same a^*-style blowup is shown to collapse back to a single equivalence class at every step once simplification is applied.

`src/regex/nullable/standard.rs` and `src/regex/deriv/standard.rs` transcribe $n(r)$ and $r\backslash x$ case for case, matching the `Regex` enum of Section 2.3 one variant at a time. `match_deriv` (`src/matchers/match_deriv.rs`) folds `deriv` over the input characters and checks `nullable` on the result, exactly as `matchDeriv` does in the reference implementation. One deliberate departure from that reference: after every derivative step, `match_deriv` also applies `simplify` (`src/regex/simplify/standard.rs`, discussed further in Section 5.4), collapsing exactly the algebraic identities used informally above – $\emptyset \cdot r = \emptyset$, $\varepsilon \cdot r = r$, $\emptyset + r = r$, and idempotent alternation $r + r = r$. Applied to the trace above, `simplify` would keep the expression at $\varepsilon \cdot x^*$ (equivalently, after normalizing $\varepsilon \cdot r$ to r, at x^* itself) after every step rather than letting it grow. Since `simplify` preserves $L(r)$ by construction, this changes nothing about which words are accepted; it only keeps the expressions produced along the way from growing as in the example above.

The sequence of derivatives $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ can be read as tracing a path through an automaton built lazily, on demand: each r_i is a state, and $\xrightarrow{x}$ is the transition relation. This "derivatives as an implicit DFA" reading is what motivates calling `match_deriv` DFA-style throughout this thesis – at every step there is exactly one current expression, playing the role of exactly one current automaton state, however large that single state's internal syntax tree happens to be.

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 keeps the residual language as a single expression, at the cost of that expression's internal structure growing with every step – the $+$ introduced by

the concatenation rule nests deeper each time a nullable prefix is derived again, and only an explicit simplification pass folds it back down. Antimirov's partial derivatives take a different route to the same residual language: rather than combining the alternatives produced along the way with $+$, keep them apart as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set,

$$
\begin{aligned}
pd(\emptyset, x) &= \{\} \\
pd(\varepsilon, x) &= \{\} \\
pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} \\
pd(r + s, x) &= pd(r, x) \cup pd(s, x) \\
pd(r^*, x) &= \{\, smartC(r', r^*) \mid r' \in pd(r, x) \,\} \\
pd(r \cdot s, x) &= \begin{cases} \{\, smartC(r', s) \mid r' \in pd(r, x) \,\} \cup pd(s, x) & n(r) \\ \{\, smartC(r', s) \mid r' \in pd(r, x) \,\} & \text{otherwise} \end{cases}
\end{aligned}
$$

where the smart constructor $smartC(\varepsilon, s) = s$ and $smartC(r', s) = r' \cdot s$ otherwise normalizes away the spurious $\varepsilon \cdot s$ that would otherwise appear whenever a partial derivative of r is exactly ε. This normalization is not just cosmetic: it is what guarantees every partial derivative is literally a subexpression of r (up to the concatenations $smartC$ builds), rather than an expression that could in principle grow without bound the way plain $r\backslash x$ can.

Worked example: derivative versus partial derivative, side by side. Take $r = a + ab$ and derive with respect to $\mathtt{a}$ under both constructions. Brzozowski's derivative, following Section 3.2, produces one expression:

$$r\backslash\mathtt{a} = (a\backslash\mathtt{a}) + (ab\backslash\mathtt{a}) = \varepsilon + ((a\backslash\mathtt{a}) \cdot b) = \varepsilon + (\varepsilon \cdot b),$$

which `simplify` would reduce to $\varepsilon + b$ – still a *single* expression, whose top-level shape is an alternation joining the two strands together. Antimirov's partial derivative keeps those same two strands apart instead of joining them with $+$:

$$
\begin{aligned}
pd(r, \mathtt{a}) &= pd(a, \mathtt{a}) \cup pd(ab, \mathtt{a}) \\
&= \{\varepsilon\} \cup \{\, smartC(r', b) \mid r' \in pd(a, \mathtt{a}) \,\} \\
&= \{\varepsilon\} \cup \{smartC(\varepsilon, b)\} \;=\; \{\varepsilon, b\}.
\end{aligned}
$$

Both describe the same residual language, $\{\varepsilon\} \cup \{\mathtt{b}w \mid w \in \Sigma^*\}$ restricted to $\{\mathtt{b}\}$ itself here – but $r\backslash\mathtt{a}$ represents it as one expression with an internal $+$, while $pd(r, \mathtt{a})$ represents it as two separate, syntactically smaller expressions with no $+$ at all. This is the concrete sense in which pd is NFA-shaped: $\{\varepsilon, b\}$ is exactly the state set an NFA for r would be in after reading $\mathtt{a}$, with one state per strand, rather than one DFA state encoding both strands internally.

That subexpression property underlies Antimirov's central finiteness result: the set of *all* partial derivatives reachable from r by derivation with respect to arbitrary words is finite, bounded by the number of literal-symbol occurrences in r [Antimirov, 1996]. For $r = a + ab$ above, there are three literal occurrences (the two a's and the one b), so at most three *distinct* partial derivatives (plus r itself) can ever appear, no matter

how long a word r is derived with respect to – and indeed $pd(r, \mathtt{a}) = \{\varepsilon, b\}$ already uses two of that budget. Ordinary derivatives are also finite up to language equivalence, but only after applying simplification laws such as those of Section 5.4 to identify equivalent expressions; the partial-derivative bound holds already at the syntactic level, with no equivalence-checking required.

`src/regex/pderiv/standard.rs` implements pd as `HashSet<Regex>`, reusing the same `smart_seq` smart constructor already introduced for `simplify` in Section 3.2 – `smartC` and `smart_seq` coincide, since both exist to normalize $\varepsilon \cdot r$ to r.

The matcher itself, `match_pderiv` in `src/matchers/match_pderiv.rs`, mirrors `matchPDeriv` from the reference implementation: rather than tracking one expression, it maintains a `HashSet<Regex>` of currently-live residuals, replacing that set at each character with the union of $pd(r', x)$ over every residual r' currently in it, and accepting once the input is exhausted if any surviving residual is nullable. Running `match_pderiv` on $r = a + ab$ against `ab` starts from $\{r\}$, steps to $pd(r, \mathtt{a}) = \{\varepsilon, b\}$ exactly as computed above, then steps again on `b`: $pd(\varepsilon, \mathtt{b}) \cup pd(b, \mathtt{b}) = \{\} \cup \{\varepsilon\} = \{\varepsilon\}$, which contains a nullable residual, so the word is accepted. This is exactly subset simulation of an NFA whose states are subexpressions of the original r, run directly on the syntax rather than on a separately constructed automaton.

3.4 Matcher Selection

`match_naive`, `match_deriv`, and `match_pderiv` share the signature `fn(&str, &Regex) -> bool` despite implementing three structurally different algorithms – exhaustive search over substring splits, a single evolving DFA-style expression, and an evolving NFA-style set of expressions. This uniform interface is what makes the three matchers directly interchangeable: the same regular expression and input can be run through any of them, and any disagreement between their answers points to a bug in one of the three independently-written implementations rather than to an ambiguity in what the "correct" answer is supposed to be. This is precisely the property the matcher-agreement tests in the test suite exploit, and the same pattern is reused throughout this thesis for parsers rather than matchers, wherever multiple independently-derived implementations are expected to agree.

`MatcherType` (`src/matchers/selection.rs`) enumerates the three matchers and gives library callers a single place to resolve "which matcher runs" without hard-coding a choice at every call site – either via the `REGEX_MATCHER` environment variable (`naive`, `deriv`, `pderiv`, or `all`, defaulting to `deriv`), used by the standalone demos under `examples/`, or, for the `regex-engine` command-line tool itself, an explicit `--matcher` flag on the `match` subcommand – the CLI does not read `REGEX_MATCHER` at all, so that there is exactly one place (the flag) resolving "which matcher" for a given invocation rather than two that could disagree. Either way, all three matchers can also be run side by side on one input in a single invocation:

```
cargo run -- match "a*" "aaa" --matcher all
```

prints the result of `match_naive`, `match_deriv`, and `match_pderiv` together, letting the three be compared directly rather than one at a time.

The POSIX parsers built in Chapters 5 and 6 follow the same pattern one level up: rather

than a boolean, each of the parser combinations introduced there returns a parse tree, but all of them are again selectable through a single uniform interface and are expected to agree with one another wherever they overlap.

Matching versus Parsing

Chapter 3 answered the membership question, always with a Boolean. This chapter motivates and makes precise the harder question the rest of this thesis is about: not just *whether* r matches w, but *how* – which parts of w were consumed by which parts of r. That question turns out not to have a unique answer in general, and this chapter's second half develops the disambiguation machinery, in particular the POSIX policy, needed to make it one again.

4.1 The Matching Problem

The three matchers of Chapter 3 all decide the same question: given r and w, is $w \in L(r)$? A Boolean answer is enough for some uses of regular expressions – validating that a password meets a policy, or that a configuration value has an acceptable shape, needs nothing more. It is not enough for others. A lexer must know not just that an input decomposes into valid tokens, but exactly where each token begins and ends. Capturing groups, schema validation, and information extraction all require identifying which *part* of w corresponds to which *part* of r, not merely that some correspondence exists. In each of these settings the question is no longer recognition but parsing: recover a witness that $w \in L(r)$, structured enough to read off submatch boundaries from, rather than a single bit reporting that such a witness exists.

4.2 Parse Trees as Matching Evidence

Sulzmann and Lu formalize such a witness by viewing regular expressions as types and parse trees as the values inhabiting them [Sulzmann and Lu, 2014]. A parse tree v is built from the same constructors used informally in Section 2.2's definition of $L(r)$ – a pair for concatenation, a tagged left or right injection for alternation, a list for repetition – and a judgment $\vdash v : r$, defined by structural recursion on both v and r simultaneously, states that v is a valid parse tree of r:

$$\vdash () : \varepsilon \qquad \frac{l \in \Sigma}{\vdash l : l}$$

$$\frac{\vdash v_1 : r_1 \quad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2} \qquad \frac{\vdash v_1 : r_1}{\vdash \mathit{Left}\ v_1 : r_1 + r_2} \qquad \frac{\vdash v_2 : r_2}{\vdash \mathit{Right}\ v_2 : r_1 + r_2}$$

$$\frac{}{\vdash [\,] : r^*} \qquad \frac{\vdash v : r \quad \vdash vs : r^*}{\vdash (v : vs) : r^*}$$

A *flattening* function $|v|$ reads the underlying word back off a parse tree by structural recursion – $|()| = \varepsilon$, $|l| = l$, $|(v_1, v_2)| = |v_1||v_2|$, $|Left\ v| = |Right\ v| = |v|$, and $|v : vs| = |v||vs|$ – and the two are related by the property that matters: for every derivable $\vdash v : r$, the word $|v|$ is a member of $L(r)$. A parse tree is thus not merely evidence that $w \in L(r)$; each derivation of $\vdash v : r$ with $|v| = w$ *is* a proof of that membership, in the same sense a value of a type is evidence that the type is inhabited.

Warm-up example. Before turning to a compound expression, it is worth seeing the rules apply to the simplest nontrivial case. For $r = a \cdot b$ against $w = \texttt{ab}$, only one derivation is possible: the *Pair* rule requires $\vdash v_1 : a$ and $\vdash v_2 : b$, and the only parse trees of a single literal are the literal itself, so $v_1 = a$, $v_2 = b$, and

$$\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}$$

is the only possible derivation, giving $v = (a, b)$ with $|v| = |a||b| = \texttt{ab}$. Nothing here is ambiguous yet – ambiguity, taken up in Section 4.3, requires a choice point such as $+$ or * somewhere in r.

Compound example. Now take $r_1 = (a + ab)(b + \varepsilon)$ against the word `ab` (`paper_r1` in the test suite, `tests/common/mod.rs`, reproducing the running example of [Sulzmann and Lu, 2014]). One valid derivation picks the second alternative of $a + ab$, matching `ab` in one step, then the empty alternative of $b + \varepsilon$:

$$\frac{\dfrac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}\,/\,\dfrac{\vdash (a, b) : a + ab}{\vdash Right(a, b) : a + ab} \qquad \dfrac{}{\vdash () : \varepsilon}\,/\,\dfrac{\vdash () : \varepsilon}{\vdash Right() : b + \varepsilon}$$

yielding $v = (Right(a, b), Right())$ with $|v| = |(a, b)||()| = \texttt{ab} \cdot \varepsilon = \texttt{ab}$. A second, equally valid derivation exists for the same r_1 and the same word – Section 4.3 constructs it explicitly, since it is exactly what makes disambiguation necessary in the first place.

`src/types/parse_tree.rs` represents v directly as the `ParseTree` enum – `Empty`, `Char`, `Pair`, `Left`, `Right`, `Star` correspond one to one with $()$, a literal, (v_1, v_2), *Left* v, *Right* v, and $v{:}vs$ respectively (Section 5.5.1 presents the Rust type itself, alongside the rest of the `inj` parser it belongs to), and `flatten` is $|v|$ by structural recursion, matching each `ParseTree` case to the corresponding clause above. One constructor present in the reference's `U` type has no counterpart here: `Nil`, used there for the (untypeable) "no parse tree" case when a match does not exist. The Rust implementation has no need for it, representing absence of a parse tree with `Option<ParseTree>::None` at the type level instead – a first, small instance of a translation choice Rust's type system invites that the untyped reference does not; Section 8.2.3 returns to this pattern more generally once the full implementation is on the table.

4.3 Ambiguity in Regular Expression Parsing

The judgment $\vdash v : r$ is not functional: for a fixed r and w, there can be more than one v with $\vdash v : r$ and $|v| = w$. Return to $r_1 = (a + ab)(b + \varepsilon)$ against `ab` from Section 4.2: alongside $v = (Right(a, b), Right())$, the *first* alternative of $a + ab$ also leads to a valid,

different derivation – match `a` against the first alternative, leaving `b` for $b + \varepsilon$'s own first alternative:

$$\frac{}{\vdash a : a} \Big/ \frac{\vdash a : a}{\vdash \mathit{Left}\ a : a + ab} \qquad \frac{}{\vdash b : b} \Big/ \frac{\vdash b : b}{\vdash \mathit{Left}\ b : b + \varepsilon}$$

giving $v' = (\mathit{Left}\ a, \mathit{Left}\ b)$, with $|v'| = |a||b| =$ `ab` – the same word, a different parse. A second example makes the same point with repetition instead of concatenation: $r_2 = (a + b + ab)^*$ against `ab` (`paper_r2`[1] in the test suite). One iteration of the star can consume `ab` in a single step via the third alternative, giving $v = [\mathit{Right}\ \mathit{Right}(a, b)]$; alternatively, two iterations can consume `a` and `b` separately via the first two alternatives, giving $v' = [\mathit{Left}\ a,\ \mathit{Right}\ \mathit{Left}\ b]$. Nothing in Section 4.2's typing rules prefers one derivation over the other in either example. A parser that must return a single answer therefore needs an explicit *disambiguation policy*: a rule that, given r and w, picks out one v from the (possibly many) parse trees satisfying $\vdash v : r$ and $|v| = w$.

4.3.1 Greedy Leftmost Matching

The policy implicit in backtracking engines is *greedy*: alternatives are tried in left-to-right order and the first one that leads to an overall match wins, and inside a Kleene star, one more iteration is always attempted before the star is allowed to stop [Frisch and Cardelli, 2004]. Operationally this is exactly what a backtracking matcher's exploration order already realizes; formalized as an ordering on parse trees, the relevant rule states that between two candidates agreeing everywhere else, $\mathit{Left}\ v \geq_{r_1+r_2} \mathit{Right}\ v'$ always – the left alternative is preferred outright, independent of how long a match each side produces.

Applied to $r_1 = (a + ab)(b + \varepsilon)$ against `ab` from above, Greedy matching tries the first alternative of $a + ab$ first: a matches the leading character, so the engine commits to it immediately without ever considering whether ab might have matched more. What remains, `b`, is then matched against $b + \varepsilon$, whose own first alternative b succeeds directly. The Greedy result is therefore $v' = (\mathit{Left}\ a, \mathit{Left}\ b)$ – the *second* of the two derivations found in Section 4.3. Section 4.3.3 revisits this exact pair $(r_1, \texttt{ab})$ once the POSIX ordering has been introduced, and finds the opposite answer.

4.3.2 The Infinite Empty-Match Chain Problem

Greedy's "always iterate before stopping" rule interacts badly with expressions that can match the empty word inside a star. Take ε^* against the empty input: since ε itself is nullable, nothing stops the star from iterating arbitrarily many times while still matching only ε overall, and Greedy's preference for "one more iteration" ranks every such expansion above the last:

$$v_0 = [\,] \ <_{\varepsilon^*}\ v_1 = [()] \ <_{\varepsilon^*}\ v_2 = [(), ()] \ <_{\varepsilon^*}\ \cdots$$

This chain has no maximum, so Greedy matching on ε^* is only well-defined once such *problematic* expressions are given special treatment beyond the ordering rule of Section 4.3.1 alone. The same issue appears in slightly less degenerate form for $(\varepsilon + a)^*$ against `a`, where $v_0 = [\mathit{Right}\ a]$, $v_1 = [\mathit{Left}\ (), \mathit{Right}\ a]$, $v_2 = [\mathit{Left}\ (), \mathit{Left}\ (), \mathit{Right}\ a]$, ...again form an infinite ascending chain under Greedy, this time padding an arbitrary number of empty iterations in front of the one iteration that actually consumes `a`. Section 4.3.3

[1] As implemented, `paper_r2` nests the alternation as $a + (b + ab)$; the associativity of $+$ is immaterial to $L(r)$ and to this example.

shows exactly how POSIX's rule (K1) blocks both chains at v_0 directly, with no special case required.

4.3.3 POSIX Longest-Leftmost Matching

The POSIX standard specifies a different policy, *leftmost-longest* matching:

> "Subpatterns should match the longest possible substrings, where subpatterns that start earlier (to the left) in the regular expression take priority over ones starting later. Hence, higher-level subpatterns take priority over their lower-level component subpatterns. Matching an empty string is considered longer than no match at all." [IEEE and The Open Group, 2017]

Sulzmann and Lu formalize this as an ordering $\geq_r$ on parse trees of a fixed r, obtained from the Greedy order of Section 4.3.1 by replacing exactly one rule and leaving the rest unchanged [Sulzmann and Lu, 2014]:

$$\text{(C1)}\quad \frac{v_1 = v_1' \quad v_2 \geq_{r_2} v_2'}{(v_1, v_2) \geq_{r_1 r_2} (v_1', v_2')}$$

$$\text{(C2)}\quad \frac{v_1 \geq_{r_1} v_1'}{(v_1, v_2) \geq_{r_1 r_2} (v_1', v_2')}$$

$$\text{(A1)}\quad \frac{\text{len}\,|v_2| > \text{len}\,|v_1|}{\mathit{Right}\ v_2 \geq_{r_1+r_2} \mathit{Left}\ v_1}$$

$$\text{(A2)}\quad \frac{\text{len}\,|v_1| \geq \text{len}\,|v_2|}{\mathit{Left}\ v_1 \geq_{r_1+r_2} \mathit{Right}\ v_2}$$

$$\text{(A3)}\quad \frac{v_2 \geq_{r_2} v_2'}{\mathit{Right}\ v_2 \geq_{r_1+r_2} \mathit{Right}\ v_2'}$$

$$\text{(A4)}\quad \frac{v_1 \geq_{r_1} v_1'}{\mathit{Left}\ v_1 \geq_{r_1+r_2} \mathit{Left}\ v_1'}$$

$$\text{(K1)}\quad \frac{|v : vs| = \varepsilon}{[\,] \geq_{r^*} v : vs}$$

$$\text{(K2)}\quad \frac{|v : vs| \neq \varepsilon}{v : vs \geq_{r^*} [\,]}$$

$$\text{(K3)}\quad \frac{v_1 \geq_r v_2}{v_1 : vs_1 \geq_{r^*} v_2 : vs_2}$$

$$\text{(K4)}\quad \frac{v_1 = v_2 \quad vs_1 \geq_{r^*} vs_2}{v_1 : vs_1 \geq_{r^*} v_2 : vs_2}$$

Congruence rules (C1), (C2) propagate a componentwise comparison through pairs: two concatenated parse trees compare by comparing each component in turn, with equality on the first component falling through to the second (C1). In place of Greedy's unconditional "left beats right", (A1) and (A2) compare by *length* first: whichever alternative produces the longer match wins outright (A1), and only when both sides match the same length does the left alternative win (A2) – recovering left-preference as a tie-breaker rather than an absolute rule. (A3), (A4) recurse structurally once a side has already been fixed. The remaining rules govern Kleene star lists: (K1) prefers the empty list $[\,]$ over any list of iterations that together still flatten to ε, and (K2) is its mirror image, preferring any non-empty-flattening list over an empty-flattening one; (K3), (K4) recurse into the list

structurally once the head elements are compared, first by the ordering on the star's body (r, via (K3)) and, only on equality, by the tail (via (K4)).

POSIX applied to the running example. Return once more to $r_1 = (a + ab)(b + \varepsilon)$ against $\texttt{ab}$, comparing $v = (\mathit{Right}(a, b), \mathit{Right}())$ against $v' = (\mathit{Left}\ a, \mathit{Left}\ b)$ from Section 4.3. Both share the same regular expression, so (C2) applies to the first component: comparing $\mathit{Right}(a, b)$ against $\mathit{Left}\ a$ within $a + ab$ is a direct instance of (A1), since $\operatorname{len}|(a, b)| = 2 > 1 = \operatorname{len}|a|$. Hence $\mathit{Right}(a, b) \geq_{a+ab} \mathit{Left}\ a$, and by (C2), $v \geq_{r_1} v'$ – POSIX selects v, the *opposite* of Greedy's answer for the same $(r_1, \texttt{ab})$ found in Section 4.3.1.

POSIX applied to the star example. For $r_2 = (a + b + ab)^*$ against $\texttt{ab}$, comparing $v = [\mathit{Right}\ \mathit{Right}(a, b)]$ against $v' = [\mathit{Left}\ a,\ \mathit{Right}\ \mathit{Left}\ b]$ starts with (K3), comparing head elements $\mathit{Right}\ \mathit{Right}(a, b)$ and $\mathit{Left}\ a$ within the star's body $a + (b + ab)$. This is again (A1): $\operatorname{len}|\mathit{Right}(a, b)| = 2 > 1 = \operatorname{len}|a|$, so $\mathit{Right}\ \mathit{Right}(a, b) \geq_{a+(b+ab)} \mathit{Left}\ a$, and (K3) lifts this to $[\mathit{Right}\ \mathit{Right}(a, b)] \geq_{r_2} [\mathit{Left}\ a,\ \mathit{Right}\ \mathit{Left}\ b]$ directly – the single two-character iteration beats the two one-character iterations under POSIX.

POSIX applied to the infinite-chain examples. Rule (K1) resolves Section 4.3.2's pathology directly: for ε^* against ε, comparing $v_0 = [\,]$ against $v_1 = [()]$ instantiates (K1) with $v = ()$, $vs = [\,]$, whose flattening $|() : [\,]| = |()|\,|[\,]| = \varepsilon \cdot \varepsilon = \varepsilon$ satisfies the rule's premise immediately, giving $[\,] \geq_{\varepsilon^*} [()]$ – and by the same argument against $v_2, v_3, \ldots$, against every element of the chain at once, with no separate case analysis needed for each one. The same instantiation of (K1) applies to $(\varepsilon + a)^*$ against $\texttt{a}$: every v_i in that chain other than $v_0 = [\mathit{Right}\ a]$ itself flattens to $\texttt{a}$ once the trailing $\mathit{Right}\ a$ is accounted for, but the comparison that matters is between v_0 and the empty-flattening prefix chains layered in front of it in $v_1, v_2, \ldots$ – and (K1) again picks the shorter, un-padded v_0 over any of them.

Unlike the Greedy order, $\geq_r$ is total and has a maximal element for *every* expression r, including the problematic ones of Section 4.3.2 – (K1) already picks out $[\,]$ as maximal for ε^* with no separate case analysis required [Sulzmann and Lu, 2014]. This single well-definedness property is what makes POSIX matching amenable to a direct algorithmic construction rather than a family of special cases, and it is exactly what Chapter 5 builds a parser around: a way to compute the $\geq_r$-maximal parse tree without materializing and comparing every candidate.

Derivative-Based Parser

Chapter 4 fixed what is being computed – the $\geq_r$-maximal parse tree of Definition 4.3.3 – without saying how. This chapter presents Sulzmann and Lu's answer [Sulzmann and Lu, 2014]: a parser built directly on the Brzozowski derivative of Section 3.2, in two forms. The first, direct construction computes a derivative sequence and then explicitly rebuilds a parse tree from it; the second, bit-coded construction fuses parse-tree bookkeeping into the derivative step itself. Both are developed here as theory first, then as the two Rust implementations that realize them, `src/posix/standard/` and `src/posix/bitcoded/`. Chapter 6 repeats the exercise once more, replacing the Brzozowski derivative throughout with Antimirov's partial derivative from Section 3.3.

5.1 Core Algorithm

5.2 Correctness Properties

5.3 Bit-Coded Optimization (Theory)

5.4 Simplification Rules

5.5 Rust Implementation: The `inj` Version

5.5.1 `ParseTree` and `flatten`

5.5.2 `mk_eps`

5.5.3 `inject`

5.5.4 `parse_recursive` vs. `parse_loop`

5.6 Rust Implementation: The Bit-Coded Version

5.6.1 `ARegex` - Bit-Annotated Regex

5.6.2 `internalize/fuse`

5.6.3 `deriv_bc`

5.6.4 `mk_eps_bc`

5.6.5 `decode`

Partial-Derivative-Based Parser

6.1 Extending POSIX Parsing to Partial Derivatives

6.2 Bit-Coded Variant

6.3 Rust Implementation

Evaluation

7.1 Compared Engines

7.1.1 Rust's `regex` Crate

7.1.2 RE2

7.2 Testing Strategy

7.3 Correctness Verification

7.3.1 Paper Examples Reproduced

7.3.2 Cross-Implementation Equivalence

7.4 Performance Analysis

7.4.1 Stack Depth: Recursive vs. Iterative Construction

7.4.2 Heap Allocation: Standard vs. Bit-Coded

7.4.3 Derivative-Based vs. Partial-Derivative-Based

7.4.4 Pathological Inputs

7.4.5 Benchmark Results

7.5 Comparison with Existing Engines

7.5.1 Performance

7.5.2 Standardized Benchmarking via rebar

Related Work and Discussion

8.1 Related Work

8.1.1 POSIX-Compliant Engines

8.1.2 RE2 and Rust's `regex` Crate

8.1.3 Other Derivative-Based Approaches

8.1.4 Kuklewicz's Observation on Ordering-Rule Ambiguity

8.2 Rust versus Haskell

8.2.1 Language Characteristics of the Reference Implementation

8.2.2 What the Reference Provides vs. What It Leaves Open

8.2.3 Translation Strategy

8.2.4 Translation Insights

8.3 Mismatch Analysis via Derivative Tracing

8.3.1 The Gap

8.3.2 Deriving Diagnostics for Free from the Algorithm Itself

8.3.3 Design

8.3.4 Scope and Limits

8.4 Optimization Challenges and Insights

Conclusion and Future Work

9.1 Summary of Contributions

9.2 Paper-Suggested Extensions Not Yet Implemented

9.2.1 Tagged NFA for Submatching

9.2.2 Space-Efficient Parsing

9.2.3 DFA/NFA Hybrid Abstraction

9.3 Further Performance Optimizations

9.4 A Full Mismatch Diagnostics System

Bibliography

Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.

Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4): 481–494, 1964. doi: 10.1145/321239.321249.

Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). https://swtch.com/~rsc/regexp/regexp1.html, 2007. Accessed: 2026-08-05.

Alain Frisch and Luca Cardelli. Greedy regular expression matching. In *Automata, Languages and Programming (ICALP 2004)*, volume 3142, pages 618–629. Springer, 2004. doi: 10.1007/978-3-540-27836-8_53.

IEEE and The Open Group. Posix.1-2017: Base definitions, chapter 9 (regular expressions), 2017. IEEE Std 1003.1-2017.

Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.

Code Listings

Full Benchmark Results

Test Case Catalogue

CLI and ENV Reference